AMLNN Toolkit Python API Guide

版本: 0.1

发布日期: 2026-05-15

版权

© 2026 晶晨半导体（上海）股份有限公司（代表其自身及其关联公司，以下简称"Amlogic"）版权所有。未经 Amlogic 书面许可，不得以任何形式或通过任何手段对本文件的任何部分进行复制、传播、转录或翻译成任何语言。

商标

Amlogic，及 Amlogic 的其他图标均为 Amlogic 公司的商标。所有其他商标和注册商标均为其各自所有人的财产。

免责声明

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

本产品不适用于医疗、救生或生命维持等用途，也不适用于可合理预见因 Amlogic 产品故障或失灵可能导致人身伤害、死亡、严重财产或环境损害的应用场景。

文中提供的电路图及其他与 Amlogic 产品相关的信息仅为典型应用说明，完整且足以用于生产设计的全部信息不一定会提供。Amlogic 已尽量确保本文件内容准确和及时，但 Amlogic 不对本文档所含信息的准确性或完整性作任何保证。

Amlogic 产品可能存在未被识别的漏洞，或可能支持具有已知局限性的既定安全标准或规范。Amlogic 对任何漏洞不承担任何责任。

使用相关材料可能需获得第三方或 Amlogic 的知识产权许可。本文档对归属于 Amlogic 或任何其他方的知识产权不授予任何明示或暗示的声明或保证。文中包含的任何第三方网站链接仅为方便用户而提供，Amlogic 不对该等网站及其任何行为（包括隐私惯例、可用性和内容）作出认可，也不承担任何责任。

在任何情况下，Amlogic 均不对与本文档相关、由本文档使用或无法使用所产生的任何索赔承担责任。

联系方式

- 公司网址：www.amlogic.cn
- 售前咨询：contact@amlogic.com
- 技术支持：support@amlogic.com

修订记录

版本	变更描述
V0.1 (2026-05-15)	初始版本。

目录

1 AMLNN API 简介

2 API说明

2.1 AMLNN初始化

2.2 模型加载

2.2.1 load_onnx

2.2.2 load_pytorch

2.2.3 load_pt2

2.2.4 load_tflite

2.3 config

2.4 compile

2.5 export_adla

2.6 init_runtime

2.7 get_sdk_version

2.8 load_model

2.9 get_tensor_info

2.10 inference

2.11 get_perf_info

2.12 perf_visualize

2.13 uninit

1 AMLNN API 简介

本文档介绍 `amlnn-toolkit`（`amlnn-edge-toolkit`）的 API 及参数说明。

本文档适配的产品如下：

- C308L / C302X
- S928X
- A311D2
- T968D4
- S905X5 / S905D5
- C302X2
- C305X2/A123X
- A311Y3

2 API说明

2.1 AMLNN初始化

在使用amlnn-toolkit的API接口时，需要先调用AMLNN()进行AMLNN对象初始化，结束时调用uninit()方法进行释放。

参数	类型	说明	必要参数
log_level	str	log 等级设置，**默认值： 'INFO'。** 当前支持['DEBUG', 'INFO', 'WARNING','ERROR', 'CRITICAL']	否

```
#Example

from amlnn.api import AMLNN

amlnn =  AMLNN(log_level='INFO')

...

amlnn.uninit()
```

2.2 模型加载

`amlnn_toolkit` 支持 ONNX、TFLite（float / quant）、PyTorch（pt / pt2）等模型的加载与转换。不同框架需调用对应接口，详细说明如下：

2.2.1 load_onnx

功能：onnx模型加载。

返回值：0：成功 ， -1：失败

参数	类型	说明	必要参数
model	str	onnx模型文件路径。	是
inputs	list[str]	onnx模型输入节点（tensor）。 仅在需要指定输入节点和形状时才设置。**默认值：None。**多输入设置方式参考：['input1','input2'] Note：**原始onnx中输入shape是动态shape时必须设置。或者需要调整原始onnx的shape尺寸时也可设置。**	否
input_shapes	list[list]	onnx模型输入尺寸。 仅在需要指定输入节点和形状时才设置。**默认值：None。**多输入设置方式参考：[[1,3,224,224],[1,3,229,229]] Note：**原始onnx中输入shape是动态shape时必须设置。或者需要调整原始onnx的shape尺寸时也可设置。**	否
outputs	list[str]	指定onnx模型输出节点名称（tensor name）。**仅在对onnx模型做截断时设置。**默认值：None，表示不做截断。	否

```
#Example1:
amlnn.load_onnx(model='./yolov8s_sim.onnx')

#Example2: 加载模型，并做截断
amlnn.load_onnx(model='./yolov8s_sim.onnx', \
        outputs=
['/model.22/Concat_output_0','/model.22/Concat_1_output_0','/model.22/Concat_2_output_0'])
```

2.2.2 load_pytorch

功能：pytorch jit模型加载。

返回值：0：成功， -1：失败

参数	类型	说明	必要参数
model	str	pytorch模型文件路径。通过torch.jit.trace保存的pt文件。	是

参数	类型	说明	必要参数
inputs	list[str]	pytorch模型输入节点名称(tensor name)。 pt模型一般没有输入shape信息，所以需要外部指定。多输入设置方式参考：['input1','input2']	是
input_shapes	list[list]	pytorch模型输入尺寸。 多输入设置方式参考：[[1,3,224,224],[1,3,229,229]]	是
dtypes	list[str]	模型输入节点类型，默认值：None。使用默认值时，所有输入默认为float32。 多输入设置方式参考：['float32','float32','int32']。 Note：**仅当模型输入节点均为 float 时，dtypes 为非必填参数；否则必须通过 dtypes 指定输入节点类型。**	否

```
#Example:
amlnn.load_pytorch(model='resnet18.pt',
                inputs=['%x.2'],
                input_shapes=[[1,3,224,224]])
```

2.2.3 load_pt2

功能：pytorch pt2模型加载**（ARM64版本暂不支持该接口）**。

返回值：0：成功 ， -1：失败

参数	类型	说明	必要参数
model	str	pytorch模型文件路径。通过torch.export.save保存的pt2文件。	是
outputs	list[str]	pt2模型输出节点名称(tensor name)。仅在对pt2模型做截断时设置。**默认值：None，表示不做截断**	否

```
#Example:
amlnn.load_pt2(model='yolov8n_exported.pt2')
```

2.2.4 load_tflite

功能：tflite模型加载。

返回值：0：成功 ， -1：失败

参数	类型	说明	必要参数
model	str	tflite模型文件路径	是

参数	类型	说明	必要参数
quantized_model	bool	指定该模型是否是量化模型。**默认值：False** 。当前tflite为量化tflite时，需要设置为True	否

```
#Example1：
amlnn.load_tflite(model='yolov8n_sim.tflite')

#Example2:
amlnn.load_tflite(model='yolov8n_sim.tflite',quantized_model=True)
```

2.3 config

功能： 配置模型转换参数

返回值：0：成功 ， -1：失败

参数	类型	说明	必要参数
normalization_mean	[list[float]]	输入各通道的均值。**默认值：None**，表示均值均为0。	否
normalization_std	[list[float]]	输入归一化值。**默认值：None**，表示标准差均为1。	否
quantized_dtype	str	量化类型，**默认值：'w8a8'**，目前支持的量化类型有['w4a8','w8a8','w4a16','w8a16','w16a16']	否
activation_dtype	str	指定激活类型，**默认值：'f16'**，目前支持['f16','i16','bf16']	否
quantized_method	str	指定量化方式，**默认值：'perchannel'**，目前支持['perlayer', 'perchannel']	否
group_size	int	子通道分组(Sub-channel-wise)量化，即将通道划分为更小的子组，以实现更细粒度的精度控制。 **默认值：32 ,**目前支持：[32,64,128] Note：只有在quantized_dtype为w4a8、w4a16时对fc层有效。	否
activation_quant_algo	str	指定激活量化算法。 **默认值'normal'：表示使用min-max量化。**支持：['normal', 'histogram', 'percentile', 'omse']	否
weight_quant_algo	str	权重量化算法，**默认值'normal'，使用min-max量化。** 支持：['normal', 'decoupleQ','lsq'].	否
histogram_bins	int	激活量化算法为'histogram'时，直方图的bin数，**默认值:2048**，有效值：[1024,2048,4096]	否
omse_range	float	激活量化算法为'omse'时，搜索范围，**默认值：90**，有效值：(0~100)	否
omse_steps	float	激活量化算法为'omse'时，搜索步长，**默认值:0.02**，有效值：(0~1)	否
percentile_sigma	float	激活量化算法为'percentile'时，sigma参数，**默认值：0.01**，有效值：(0~1)	否
percentile_alpha	float	激活量化算法为'percentile'时，alpha参数，**默认值：0.99999**，有效值：(0~1)	否
hybrid_ratio	float	混合量化百分比。w4a8、w4a16 量化时，控制卷积 4bit 量化的比例。 **默认值：0.5**，有效值：(0~1)	否
apply_smooth_o3	bool	指定是否使用smooth算法。**默认值：False**，有效值：[True,False]	否
inference_input_type	str	指定导出ADLA的输入类型，**默认值："quant"（与转换类型一致）**，quantized_type为'w4a8','w8a8'时，支持设置['int8','uint8','float32']. quantized_type='w8a16',activation_dtype='i16'时，支持设置['int16','float32']	否
inference_output_type	str	指定导出ADLA的输出类型，**默认值："quant"（与转换类型一致）**，quantized_type为'w4a8','w8a8'时，支持设置['int8','uint8','float32']. quantized_type='w8a16',activation_dtype='i16'时，支持设置['int16','float32']	否
export_intermediate	bool	指定是否将转换中间IR导出成tflite模型文件。**默认值：False**	否
output_dir	str	输出路径，**默认值：None，表示默认输出路径为：model_name + "_adla_unify"**	否
target_platform	str	指定平台类型。目前支持： ["PRODUCT_PID0XA001", "PRODUCT_PID0XA002", "PRODUCT_PID0XA003", "PRODUCT_PID0XA004", "PRODUCT_PID0XA005","PRODUCT_PID0XA006","PRODUCT_PID0XA007","PRODUCT_PID0XA008"]	是

```
#Example
amlnn.config(target_platform='PRODUCT_PID0XA007')
```

2.4 compile

功能： 编译AMLNN模型。

返回值：0：成功， -1：失败

参数	类型	说明	必要参数
dataset	str	量化校准数据集，设置dataset.txt文件路径（文件中提供图片或npy路径），**默认值：None**，表示使用随机数进行量化，仅供测试性能，无法保证精度。 一般建议提供200~500张图片或者npy数据集。dataset.txt中放置文件路径信息，具体参考如下： 单输入模型dataset.txt(图片或.npy文件均可)： 1.jpg 2.jgp 多输入模型dataset.txt（一行放一组输入，中间用空格隔开即可）： input1_1.jpg input2_1.jpg input1_2.jpg input2_2.jpg 或者 input1_1.npy input2_1.npy input1_2.npy input2_2.npy Note：使用npy文件做量化数据时，不会使用config中配置的mean和std参数，所以需做完前处理再保存成npy，且npy的shape 需要跟input_shapes参数设置的保持一致。	否
iterations	int	指定量化所用的数据样本数（图片或 npy 文件），从 `dataset.txt` 中读取。**默认值：-1**，表示遍历全部数据；当 `dataset` 为 `None` 时，仅生成一组随机数据用于量化	否

```
#Example
amlnn.compile(dataset='dataset.txt', iterations=200)
```

2.5 export_adla

功能： 导出adla模型文件。

返回值：0：成功 ， -1：失败

```
#Example
amlnn.export_adla()
```

2.6 init_runtime

功能： 初始化运行环境。

返回值：0：成功 ， -1：失败

参数	类型	说明	必要参数
mode	str	运行模式，**默认值：'nnserver'**， 当前支持['native','nnserver']。	否
board_work_path	str	板端工作目录，默认值为 `/data/nn`。	否
enable_perf	bool	是否开启性能采集，**默认值：False，**如需获取性能信息必须设置为 `True`。	否

```
#Example
ret = amlnn.init_runtime()
```

2.7 get_sdk_version

功能： 获取当前板端 SDK / ADLA / DELEGATE 相关版本信息

返回值：`str`，版本字符串

```
#Example:
sdk_version = amlnn.get_sdk_version()
```

2.8 load_model

功能： 加载runtime运行模型。

返回值：0：成功 ， -1：失败

参数	类型	说明	必要参数
path	str	本地模型路径，目前仅支持 `.adla`。 Note：完成模型转换并执行amlnn.init_runtime后，不需要再显式调用load_model。	是

```
#Example
amlnn.load_model(path="./adla/resnet18-v1-7_int8_A311D2.adla")
```

2.9 get_tensor_info

功能： 获取模型输入输出tensor信息。

返回值：dict，包含如下两个字段：

- `inputs`：模型输入 tensor 信息列表
- `outputs`：模型输出 tensor 信息列表

```
#Examples:
tensor_info = amlnn.get_tensor_info()
```

每个 tensor 信息均为一个 `dict`，字段说明如下：

字段名	类型	说明
index	int	当前 tensor 在输入或输出列表中的索引
name	str	tensor 名称
n_dims	int	tensor 维度个数
dims	tuple	tensor 原生 shape，顺序与 `format_name` 对应
n_elems	int	tensor 元素总数
size	int	tensor 原始有效数据大小，单位为字节
format	int	tensor 布局格式的内部枚举值
format_name	str	tensor 布局格式，如 `NHWC` / `NCHW`
type	int	tensor 数据类型的内部枚举值
type_name	str	tensor 数据类型名称，如 `INT8`、`UINT8`、`FP32` 等
zp	int	量化 zero point
scale	float	量化 scale

2.10 inference

功能：执行模型推理。

返回值：返回输出 tensor 列表

- `list[np.ndarray]`
- 返回的输出 tensor 数据为 `float32` 格式

参数	类型	说明	必要参数
inputs	list/tuple/np.ndarray	输入数据，可传单个 `numpy.ndarray`，也可传 `list` / `tuple`	是
inputs_data_format	str	输入数据格式，支持 `NHWC` / `NCHW`，默认值为 `NHWC`	否

参数	类型	说明	必要参数
outputs_data_format	str	输出数据格式，支持 `NHWC` / `NCHW`，默认值为 `NHWC`	否
run_cycles	int	推理循环次数，默认值为 `1`	否

```
#Example
outputs = amlnn.inference(
    inputs=[input_data],
    inputs_data_format="NHWC",
    outputs_data_format="NHWC",
    run_cycles=1,
)
```

2.11 get_perf_info

功能：获取当前模型最近一次推理对应的性能信息。

返回值：`str`，格式化后的性能信息字符串，字段包括：-

- `npu_avg_time_ms`
- `fps`
- `memory_usage_mb`
- `bandwidth_mb`
- `elapsed_in_hw_op_ms`
- `elapsed_in_sw_op_ms`

该接口的前置条件：

- 调用 `init_runtime()` 时必须设置 `enable_perf=True`
- 调用 `get_perf_info()` 前，必须先成功调用 `inference()`

```
#Example
perf_info = amlnn.get_perf_info()
print(perf_info)

#输出结果：
npu_avg_time_ms=9.624, fps=103.907, memory_usage_mb=5.180, bandwidth_mb=28.802,
elapsed_in_hw_op_ms=5.499, elapsed_in_sw_op_ms=0.000
```

字段名	说明
npu_avg_time_ms	NPU 平均推理耗时，单位为毫秒；当 `run_cycles > 1` 时返回多轮平均值
fps	基于 `npu_avg_time_ms` 计算得到的帧率

字段名	说明
memory_usage_mb	当前模型推理所需内存占用，单位为 MB
bandwidth_mb	当前模型推理涉及的总带宽读写量，单位为 MB
elapsed_in_hw_op_ms	硬件算子总耗时，单位为毫秒
elapsed_in_sw_op_ms	软件算子总耗时，单位为毫秒

2.12 perf_visualize

功能：根据最近一次获取到的性能明细生成可视化 HTML 报告。

可视化结果：在成功运行后，会在当前工作目录下生成一个与模型名相同的文件夹，文件夹内包含如下文件：

- `profile_info.txt`
- `hard_op_chart.html`
- `soft_op_chart.html`
- `dram_rd_chart.html`
- `dram_wr_chart.html`
- `pie_charts_distribution.html`

该接口的前置条件：

- 调用 `perf_visualize()` 前，必须先成功调用 `get_perf_info()`。

```
#Example
amlnn.perf_visualize()
```

文件名	说明
profile_info.txt	原始性能明细文本，包含逐层性能数据，是生成各类可视化图表的输入文件
hard_op_chart.html	硬件算子耗时图，便于分析 NPU 侧各层耗时分布
soft_op_chart.html	软件算子耗时图，便于分析 CPU 部分的耗时分布
dram_rd_chart.html	各层 DRAM 读带宽图
dram_wr_chart.html	各层 DRAM 写带宽图
pie_charts_distribution.html	统计类饼图，展示耗时与带宽的整体分布情况

2.13 uninit

功能：释放运行时与模型资源。

```
#Example
amlnn.uninit()
```